

The Complete DSA Handbook

The Complete DSA Handbook — From Zero to Confident

PART 1: SORTING ALGORITHMS

- 1.1 Bubble Sort
- 1.2 Selection Sort
- 1.3 Insertion Sort
- 1.4 Merge Sort
- 1.5 Quick Sort
- 1.6 Heap Sort
- 1.7 Counting Sort (Non-comparison based)
- 1.8 Radix Sort
- 1.9 Bucket Sort

PART 2: SEARCHING ALGORITHMS

- 2.1 Linear Search
- 2.2 Binary Search
- 2.3 Ternary Search

PART 3: ARRAYS & STRINGS — CORE PATTERNS

- 3.1 Two Pointers
- 3.2 Sliding Window
- 3.3 Prefix Sum
- 3.4 Kadane's Algorithm (Maximum Subarray Sum)

PART 4: RECURSION & BACKTRACKING

- 4.1 Subsets
- 4.2 Permutations
- 4.3 N-Queens (classic backtracking with pruning)

PART 5: LINKED LISTS

- 5.1 Traversal
- 5.2 Reverse a Linked List
- 5.3 Detect Cycle (Floyd's Fast & Slow Pointers)
- 5.4 Merge Two Sorted Linked Lists

PART 6: STACKS & QUEUES

- 6.1 Stack (LIFO — Last In, First Out)
- 6.2 Queue (FIFO — First In, First Out)
- 6.3 Monotonic Stack (Next Greater Element)
- 6.4 Circular Queue

PART 7: TREES

- 7.1 Tree Traversals
- 7.2 BST Insert & Search
- 7.3 Segment Tree (Range Queries)
- 7.4 Fenwick Tree / Binary Indexed Tree (BIT)

PART 8: HEAPS / PRIORITY QUEUES

PART 9: GRAPHS

- 9.1 BFS (Breadth-First Search)
- 9.2 DFS (Depth-First Search)
- 9.3 Dijkstra's Algorithm (Shortest Path, non-negative weights)
- 9.4 Bellman-Ford (handles negative weights, detects negative cycles)
- 9.5 Floyd-Warshall (All-Pairs Shortest Path)
- 9.6 Union-Find (Disjoint Set) — used for Kruskal's MST & cycle detection
- 9.7 Kruskal's Algorithm (Minimum Spanning Tree)
- 9.8 Prim's Algorithm (Minimum Spanning Tree)
- 9.9 Topological Sort (Kahn's Algorithm — BFS based)

PART 10: HASHING

PART 11: DYNAMIC PROGRAMMING (DP)

- 11.1 0/1 Knapsack
- 11.2 Longest Common Subsequence (LCS)
- 11.3 Longest Increasing Subsequence (LIS)
- 11.4 Coin Change (Unbounded Knapsack style)

PART 12: GREEDY ALGORITHMS

- 12.1 Activity Selection
- 12.2 Fractional Knapsack

PART 13: BIT MANIPULATION

- 13.1 Count Set Bits
- 13.2 Single Number (find the element that doesn't repeat)

PART 14: TRIE (PREFIX TREE)

Final Advice — How to Use This Handbook

The Complete DSA Handbook — From Zero to Confident

This is your one-stop reference. For every topic, you'll get: **the idea explained simply, when to use it, working code, time/space complexity, and 5 problems to practice.** Read the idea first — always understand *why* before you memorize *how*.

PART 1: SORTING ALGORITHMS

Sorting means arranging elements in order (ascending/descending). Two things matter for every sorting algorithm: - **Stable?** Does it preserve the relative order of equal elements? (Matters when sorting objects by one field but wanting ties broken by original order.) - **In-place?** Does it need extra memory, or does it sort within the original array?

1.1 Bubble Sort

Idea (like a beginner would think of it): Repeatedly go through the array, and every time you see two adjacent elements in the wrong order, swap them. The largest element “bubbles up” to the end after each pass. Keep repeating until no swaps are needed.

When to use: Almost never in production — it’s taught for understanding, not performance. Only fine for tiny arrays or teaching purposes.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        # After each pass, the last i elements are already sorted
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped: # already sorted, stop early
            break
    return arr
```

Time: Best $O(n)$ (already sorted, with the swapped flag) | Average $O(n^2)$ | Worst $O(n^2)$ **Space:** $O(1)$ | **Stable:** Yes

1.2 Selection Sort

Idea: Find the smallest element in the unsorted part of the array and swap it to the front. Repeat for the remaining unsorted part.

When to use: When memory writes are expensive — Selection Sort does the minimum number of swaps ($O(n)$) even though comparisons are $O(n^2)$.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Time: Best/Average/Worst all $O(n^2)$ — it always scans the full remaining array regardless of order. **Space:** $O(1)$ | **Stable:** No (swapping can change relative order of equal elements)

1.3 Insertion Sort

Idea: Think of how you sort playing cards in your hand — you pick the next card and insert it into the correct position among the cards you’re already holding. Build the sorted array one element at a time.

When to use: Small arrays, or **nearly sorted** data — it’s genuinely fast (near $O(n)$) when the input is almost in order. Also used inside Quick Sort/Merge Sort for small sub-arrays as an optimization.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        # shift elements greater than key one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

Time: Best $O(n)$ (nearly sorted) | Average $O(n^2)$ | Worst $O(n^2)$ **Space:** $O(1)$ | **Stable:** Yes

1.4 Merge Sort

Idea (Divide & Conquer): Split the array into two halves, sort each half recursively, then **merge** the two sorted halves back together. The merging step is the key trick — merging two already-sorted arrays takes only $O(n)$.

When to use: When you need **guaranteed** $O(n \log n)$ performance (no worst-case blowup like Quick Sort), and stability matters. Great for linked lists and external sorting (huge files that don't fit in memory).

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]: # <= keeps it stable
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Time: Best/Average/Worst all $O(n \log n)$ **Space:** $O(n)$ (needs extra arrays for merging) | **Stable:** Yes

1.5 Quick Sort

Idea (Divide & Conquer): Pick a **pivot** element. Partition the array so everything smaller than the pivot goes to its left, everything bigger goes to its right. Recursively sort the left and right parts. Unlike Merge Sort, the “hard work” happens during partitioning, not merging.

When to use: The default choice in most language libraries (with optimizations) because it's fast in practice and sorts in-place (less memory than Merge Sort). Avoid if worst-case guarantees matter (e.g., real-time systems) — a bad pivot choice on already-sorted data can degrade to $O(n^2)$.

```
def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        pivot_index = partition(arr, low, high)
        quick_sort(arr, low, pivot_index - 1)
        quick_sort(arr, pivot_index + 1, high)
    return arr

def partition(arr, low, high):
    pivot = arr[high] # choosing last element as pivot
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

Time: Best/Average $O(n \log n)$ | Worst $O(n^2)$ (bad pivot on sorted/reverse-sorted data — mitigated with random pivot) **Space:** $O(\log n)$ (recursion stack) | **Stable:** No

1.6 Heap Sort

Idea: Build a **max-heap** from the array (a structure where the parent is always $\geq$ its children). The largest element is now at the root — swap it to the end of the array, shrink the heap by 1, and “heapify” again. Repeat until sorted.

When to use: When you need guaranteed $O(n \log n)$ **and** $O(1)$ extra space (unlike Merge Sort) — good for memory-constrained systems. Also the basis of priority queues.

```
def heap_sort(arr):
    n = len(arr)

    # Build max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0] # move max to the end
        heapify(arr, i, 0) # heapify the reduced heap
    return arr

def heapify(arr, n, i):
    largest = i
    left, right = 2 * i + 1, 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest) # heapify the affected subtree
```

Time: Best/Average/Worst all $O(n \log n)$ **Space:** $O(1)$ | **Stable:** No

1.7 Counting Sort (Non-comparison based)

Idea: Instead of comparing elements, **count** how many times each value appears (works only for integers in a known, small range). Then rebuild the array using those counts. This breaks the $O(n \log n)$

comparison-sort lower bound because it never compares two elements directly.

When to use: When values are integers in a small, known range (e.g., ages 0-120, exam scores 0-100). Terrible if the range is huge (e.g., sorting 10 random numbers between 1 and 10 billion).

```
def counting_sort(arr, max_val):
    count = [0] * (max_val + 1)
    for num in arr:
        count[num] += 1

    result = []
    for value, freq in enumerate(count):
        result.extend([value] * freq)
    return result
```

Time: $O(n + k)$ where k = range of input values **Space:** $O(k)$ | **Stable:** Yes (in the array-based version above; the prefix-sum version is also stable)

1.8 Radix Sort

Idea: Sort numbers digit by digit, starting from the **least significant digit** to the most significant, using a stable sort (usually Counting Sort) at each digit position. After processing all digits, the array is fully sorted.

When to use: Sorting large lists of integers or fixed-length strings when the number of digits (d) is small compared to $\log(n)$ — e.g., sorting phone numbers, IDs.

```
def radix_sort(arr):
    if not arr:
        return arr
    max_val = max(arr)
    exp = 1
    while max_val // exp > 0:
        counting_sort_by_digit(arr, exp)
        exp *= 10
    return arr

def counting_sort_by_digit(arr, exp):
    n = len(arr)
    output = [0] * n
    count = [0] * 10

    for num in arr:
        digit = (num // exp) % 10
        count[digit] += 1

    for i in range(1, 10):
        count[i] += count[i - 1]

    for i in range(n - 1, -1, -1):  # iterate backward to keep it stable
        digit = (arr[i] // exp) % 10
        output[count[digit] - 1] = arr[i]
        count[digit] -= 1

    for i in range(n):
        arr[i] = output[i]
```

Time: $O(d \times (n + k))$ where d = number of digits, k = base (10)

Space: $O(n + k)$ | **Stable:** Yes

1.9 Bucket Sort

Idea: Distribute elements into a number of “buckets” based on value range, sort each bucket individually (usually with Insertion Sort since buckets are small), then concatenate all buckets in order.

When to use: When input is **uniformly distributed** over a range (e.g., random floats between 0 and 1). Performs poorly if data clusters into a few buckets.

```
def bucket_sort(arr, bucket_count=10):
    if not arr:
        return arr
    max_val, min_val = max(arr), min(arr)
    bucket_range = (max_val - min_val) / bucket_count + 1e-9
    buckets = [[] for _ in range(bucket_count)]

    for num in arr:
        idx = int((num - min_val) / bucket_range)
        buckets[idx].append(num)

    result = []
    for bucket in buckets:
        result.extend(sorted(bucket)) # insertion sort in practice for small buckets
    return result
```

Time: Average $O(n + k)$ | Worst $O(n^2)$ (all elements in one bucket)
Space: $O(n + k)$ | **Stable:** Yes (if the sub-sort used is stable)

✦ **Sorting — Quick Comparison Table**

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Yes
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	Yes

🔖 **Top 5 Sorting Problems to Practice**

1. Sort Colors (Dutch National Flag Problem)
2. Merge Intervals
3. Kth Largest Element in an Array
4. Meeting Rooms II
5. Relative Sort Array

PART 2: SEARCHING ALGORITHMS

2.1 Linear Search

Idea: Check every element one by one until you find the target. The simplest possible search.

When to use: Unsorted data, or small datasets where setting up a sorted structure isn't worth it.

```
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1
```

Time: $O(n)$ | **Space:** $O(1)$

2.2 Binary Search

Idea: Only works on **sorted** data. Check the middle element — if it's the target, done. If the target is smaller, search the left half; if larger, search the right half. You throw away half the search space every step.

When to use: Any time data is sorted (or you can sort it once and query many times), or you're searching over a monotonic "answer space" (see Modified Binary Search pattern).

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

# Recursive version
def binary_search_recursive(arr, target, lo, hi):
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, hi)
    else:
        return binary_search_recursive(arr, target, lo, mid - 1)
```

Variant — Find first/last occurrence (when duplicates exist):

```
def find_first(arr, target):
    lo, hi, result = 0, len(arr) - 1, -1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            result = mid
            hi = mid - 1  # keep searching left for an earlier occurrence
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return result
```

Time: $O(\log n)$ | **Space:** $O(1)$ iterative, $O(\log n)$ recursive (call stack)

2.3 Ternary Search

Idea: Like binary search but splits the range into **three** parts using two midpoints. Mainly used to find the maximum/minimum of a unimodal function (one that strictly increases then strictly decreases, or vice versa) — not for searching a plain sorted array (binary search is better there).

```
def ternary_search(f, lo, hi, epsilon=1e-9):
    while hi - lo > epsilon:
        m1 = lo + (hi - lo) / 3
        m2 = hi - (hi - lo) / 3
        if f(m1) < f(m2):
            lo = m1
        else:
            hi = m2
    return lo # point where f is maximized
```

Time: $O(\log_3 n)$ — same order as binary search, slightly more comparisons per step

🔗 Top 5 Searching Problems to Practice

1. Search in Rotated Sorted Array
 2. Find First and Last Position of Element in Sorted Array
 3. Median of Two Sorted Arrays
 4. Search a 2D Matrix
 5. Koko Eating Bananas (binary search on answer)
-

PART 3: ARRAYS & STRINGS — CORE PATTERNS

3.1 Two Pointers

Idea: Use two indices moving through the array (toward each other or in the same direction) to avoid nested loops.

```
def two_sum_sorted(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        curr = arr[left] + arr[right]
        if curr == target:
            return [left, right]
        elif curr < target:
            left += 1
        else:
            right -= 1
    return [-1, -1]
```

Time: $O(n)$ | **Space:** $O(1)$

3.2 Sliding Window

Idea: Maintain a “window” over a contiguous section of the array/string; expand and shrink it based on a condition, instead of recomputing from scratch each time.

```
def max_sum_subarray_k(arr, k):
    window_sum = sum(arr[:k])
    max_sum = window_sum
    for i in range(k, len(arr)):
        window_sum = window_sum - arr[i-k] + arr[i]
```

```

window_sum += arr[i] - arr[i - k] # slide the window
max_sum = max(max_sum, window_sum)
return max_sum

```

Time: $O(n)$ | **Space:** $O(1)$

3.3 Prefix Sum

Idea: Precompute cumulative sums so any range-sum query becomes $O(1)$ instead of $O(n)$.

```

def build_prefix_sum(arr):
    prefix = [0] * (len(arr) + 1)
    for i, num in enumerate(arr):
        prefix[i + 1] = prefix[i] + num
    return prefix

def range_sum(prefix, l, r): # sum of arr[l..r] inclusive
    return prefix[r + 1] - prefix[l]

```

Time: $O(n)$ build, $O(1)$ per query | **Space:** $O(n)$

3.4 Kadane's Algorithm (Maximum Subarray Sum)

Idea: Walk through the array keeping a running sum. If the running sum ever goes negative, it can only hurt future sums — so reset it to 0 (start fresh from the next element). Track the best sum seen.

```

def kadane(arr):
    max_ending_here = max_so_far = arr[0]
    for num in arr[1:]:
        max_ending_here = max(num, max_ending_here + num)
        max_so_far = max(max_so_far, max_ending_here)
    return max_so_far

```

Time: $O(n)$ | **Space:** $O(1)$

🔖 Top 5 Array/String Problems to Practice

1. Maximum Subarray (Kadane's)
2. Longest Substring Without Repeating Characters
3. Product of Array Except Self
4. Trapping Rain Water
5. 3Sum

PART 4: RECURSION & BACKTRACKING

The core idea of recursion: A function that calls itself to solve smaller versions of the same problem, until it hits a **base case** it can answer directly. Every recursive call goes on the “call stack” — think of it as a stack of unfinished tasks waiting for their sub-task to return an answer.

```

def factorial(n):
    if n == 0: # base case — stops the recursion
        return 1

```

```
return n * factorial(n - 1) # recursive case
```

Backtracking = recursion + trying every choice + **undoing** the choice if it doesn't work out (explore, then un-explore).

4.1 Subsets

```
def subsets(nums):
    result = []
    def backtrack(start, path):
        result.append(path[:]) # record current subset
        for i in range(start, len(nums)):
            path.append(nums[i]) # choose
            backtrack(i + 1, path) # explore
            path.pop() # un-choose (backtrack)
        backtrack(0, [])
    return result
```

4.2 Permutations

```
def permutations(nums):
    result = []
    def backtrack(path, remaining):
        if not remaining:
            result.append(path[:])
            return
        for i in range(len(remaining)):
            path.append(remaining[i])
            backtrack(path, remaining[:i] + remaining[i+1:])
            path.pop()
        backtrack([], nums)
    return result
```

4.3 N-Queens (classic backtracking with pruning)

```
def solve_n_queens(n):
    result = []
    cols, diag1, diag2 = set(), set(), set()
    board = []

    def backtrack(row):
        if row == n:
            result.append(board[:])
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue # pruning - skip invalid placements early
            cols.add(col); diag1.add(row - col); diag2.add(row + col)
            board.append(col)

            backtrack(row + 1)

            cols.remove(col); diag1.remove(row - col); diag2.remove(row + col)
            board.pop()

        backtrack(0)
    return result
```

Time: Backtracking is generally exponential — $O(2^n)$ for subsets, $O(n!)$ for permutations. Pruning (like in N-Queens) cuts this down drastically in practice. **Space:** $O(n)$ for recursion depth + $O(\text{result size})$ to store answers.

🔖 Top 5 Recursion/Backtracking Problems to Practice

1. Subsets
2. Permutations
3. N-Queens

4. Word Search
 5. Combination Sum
-

PART 5: LINKED LISTS

Idea: Unlike an array, a linked list stores elements as **nodes**, each holding a value and a pointer to the next node. No contiguous memory needed — insertion/deletion is $O(1)$ if you already have the pointer, but access is $O(n)$ (no random indexing like arrays).

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

5.1 Traversal

```
def traverse(head):
    curr = head
    while curr:
        print(curr.val)
        curr = curr.next
```

5.2 Reverse a Linked List

```
def reverse_list(head):
    prev = None
    curr = head
    while curr:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node
    return prev  # new head
```

Time: $O(n)$ | **Space:** $O(1)$

5.3 Detect Cycle (Floyd's Fast & Slow Pointers)

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow == fast:
        return True
    return False
```

Time: $O(n)$ | **Space:** $O(1)$

5.4 Merge Two Sorted Linked Lists

```
def merge_two_lists(l1, l2):
    dummy = ListNode()
    tail = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next, l1 = l1, l1.next
        else:
            tail.next, l2 = l2, l2.next
        tail = tail.next
```

```
tail.next = l1 or l2
return dummy.next
```

Time: $O(n + m)$ | **Space:** $O(1)$

Doubly Linked List — same as above but each node also has a `prev` pointer, so you can traverse backward and delete a node in $O(1)$ without needing the previous node passed in separately. Used in LRU Cache implementations.

🔗 Top 5 Linked List Problems to Practice

1. Reverse Linked List
2. Merge Two Sorted Lists
3. Linked List Cycle II (find the start of the cycle)
4. Remove Nth Node From End of List
5. LRU Cache

PART 6: STACKS & QUEUES

6.1 Stack (LIFO — Last In, First Out)

Idea: Like a stack of plates — you can only add/remove from the top.
Used for: undo functionality, parsing expressions, DFS, function call management.

```
stack = []
stack.append(10)    # push
stack.append(20)
top = stack[-1]     # peek
stack.pop()         # pop
```

Classic use – Valid Parentheses:

```
def is_valid(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

Time: $O(n)$ | **Space:** $O(n)$

6.2 Queue (FIFO — First In, First Out)

Idea: Like a line at a checkout counter — first person in line is served first. Used for BFS, task scheduling.

```
from collections import deque
queue = deque()
queue.append(10)      # enqueue
queue.append(20)
front = queue[0]      # peek
queue.popleft()       # dequeue - O(1), unlike list.pop(0) which is O(n)
```

6.3 Monotonic Stack (Next Greater Element)

Idea: Keep the stack in increasing/decreasing order. When a new element breaks the order, pop and resolve — each pop finalizes an answer for the popped element.

```
def next_greater_element(arr):
    result = [-1] * len(arr)
    stack = [] # stores indices
    for i, num in enumerate(arr):
        while stack and arr[stack[-1]] < num:
            idx = stack.pop()
            result[idx] = num
            stack.append(i)
    return result
```

Time: $O(n)$ — each element pushed/popped once | **Space:** $O(n)$

6.4 Circular Queue

Idea: A fixed-size queue that wraps around using modulo arithmetic, avoiding wasted space that a regular array-based queue would leave behind after dequeues.

```
class CircularQueue:
    def __init__(self, k):
        self.queue = [None] * k
        self.head = self.count = 0
        self.capacity = k

    def enqueue(self, val):
        if self.count == self.capacity:
            return False
        self.queue[(self.head + self.count) % self.capacity] = val
        self.count += 1
        return True

    def dequeue(self):
        if self.count == 0:
            return False
        self.head = (self.head + 1) % self.capacity
        self.count -= 1
        return True
```

🕒 Top 5 Stack/Queue Problems to Practice

1. Valid Parentheses
2. Min Stack
3. Next Greater Element I/II
4. Sliding Window Maximum
5. Implement Queue using Stacks

PART 7: TREES

Idea: A tree is a hierarchical structure — one root node, and each node can have children. A **Binary Tree** limits each node to at most 2 children (left, right). A **Binary Search Tree (BST)** additionally requires: left subtree < node < right subtree — this ordering is what makes search $O(\log n)$ (when balanced).

```
class TreeNode:
```

```
def __init__(self, val=0, left=None, right=None):
    self.val = val
    self.left = left
    self.right = right
```

7.1 Tree Traversals

```
def preorder(node, result=None):    # Node → Left → Right
    if result is None: result = []
    if node:
        result.append(node.val)
        preorder(node.left, result)
        preorder(node.right, result)
    return result

def inorder(node, result=None):    # Left → Node → Right (gives sorted order for BST!)
    if result is None: result = []
    if node:
        inorder(node.left, result)
        result.append(node.val)
        inorder(node.right, result)
    return result

def postorder(node, result=None):  # Left → Right → Node
    if result is None: result = []
    if node:
        postorder(node.left, result)
        postorder(node.right, result)
        result.append(node.val)
    return result
```

Time: $O(n)$ for all traversals | **Space:** $O(h)$, h = tree height (recursion stack)

7.2 BST Insert & Search

```
def insert_bst(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left = insert_bst(root.left, val)
    else:
        root.right = insert_bst(root.right, val)
    return root

def search_bst(root, val):
    if not root or root.val == val:
        return root
    return search_bst(root.left, val) if val < root.val else search_bst(root.right, val)
```

Time: $O(h)$ — $O(\log n)$ if balanced, $O(n)$ if skewed (worst case, like a linked list) | **Space:** $O(h)$

AVL Tree (concept): A **self-balancing** BST. After every insert/delete, it checks the “balance factor” (height difference between left/right subtrees) and performs **rotations** if it exceeds 1, guaranteeing $O(\log n)$ height always. You rarely implement this from scratch in interviews — just know it exists to fix BST’s worst-case skew problem.

7.3 Segment Tree (Range Queries)

Idea: A tree structure that lets you answer range queries (sum/min/max over a range) **and** update elements, both in $O(\log n)$ — much better than $O(n)$ per query with a plain array.

```
class SegmentTree:
    def __init__(self, arr):
        self.n = len(arr)
```

```

self.tree = [0] * (2 * self.n)
for i in range(self.n):
    self.tree[self.n + i] = arr[i]
for i in range(self.n - 1, 0, -1):
    self.tree[i] = self.tree[2*i] + self.tree[2*i+1]

def update(self, pos, value):
    pos += self.n
    self.tree[pos] = value
    while pos > 1:
        pos //= 2
        self.tree[pos] = self.tree[2*pos] + self.tree[2*pos+1]

def query(self, l, r): # sum of range [l, r)
    result = 0
    l += self.n; r += self.n
    while l < r:
        if l % 2 == 1:
            result += self.tree[l]; l += 1
        if r % 2 == 1:
            r -= 1; result += self.tree[r]
        l //= 2; r //= 2
    return result

```

Time: $O(\log n)$ query/update, $O(n)$ build | **Space:** $O(n)$

7.4 Fenwick Tree / Binary Indexed Tree (BIT)

Idea: A more compact structure than a Segment Tree, specifically for **prefix sums** with updates. Uses clever bit tricks ($i \& -i$) to jump between related indices.

```

class FenwickTree:
    def __init__(self, n):
        self.tree = [0] * (n + 1)

    def update(self, i, delta):
        while i < len(self.tree):
            self.tree[i] += delta
            i += i & (-i) # move to next relevant index

    def prefix_sum(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & (-i) # move to parent
        return total

```

Time: $O(\log n)$ update/query | **Space:** $O(n)$

🔗 Top 5 Tree Problems to Practice

1. Validate Binary Search Tree
2. Lowest Common Ancestor of a Binary Tree
3. Binary Tree Level Order Traversal
4. Diameter of Binary Tree
5. Serialize and Deserialize Binary Tree

PART 8: HEAPS / PRIORITY QUEUES

Idea: A heap is a complete binary tree where the parent is always $\geq$ children (max-heap) or $\leq$ children (min-heap). It's stored efficiently as an array (no pointers needed) — for index i , children are at $2i+1$ and

$2i+2$. This gives $O(\log n)$ insert/remove of the min/max, and $O(1)$ peek.

```
import heapq

min_heap = []
heapq.heappush(min_heap, 5)
heapq.heappush(min_heap, 2)
heapq.heappush(min_heap, 8)
smallest = heapq.heappop(min_heap) # 2

# Python only has min-heap - simulate max-heap by negating values
max_heap = []
heapq.heappush(max_heap, -5)
largest = -heapq.heappop(max_heap)
```

Building a heap from scratch (heapify) — see Heap Sort above for the `heapify` function.

Time: Insert/Delete $O(\log n)$, Peek $O(1)$, Build heap from array $O(n)$

Space: $O(n)$

🔗 Top 5 Heap Problems to Practice

1. Kth Largest Element in an Array
2. Find Median from Data Stream
3. Merge K Sorted Lists
4. Top K Frequent Elements
5. Task Scheduler

PART 9: GRAPHS

Idea: A graph is a set of nodes (vertices) connected by edges — can be directed/undirected, weighted/unweighted. Represented as:

```
# Adjacency List (most common - space efficient for sparse graphs)
graph = {0: [1, 2], 1: [2], 2: [0, 3], 3: [3]}

# Adjacency Matrix (good for dense graphs, O(1) edge lookup)
matrix = [[0,1,1,0],[0,0,1,0],[1,0,0,1],[0,0,0,1]]
```

9.1 BFS (Breadth-First Search)

Idea: Explore level by level using a queue. Guarantees shortest path in an **unweighted** graph.

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order
```

Time: $O(V + E)$ | **Space:** $O(V)$

9.2 DFS (Depth-First Search)

Idea: Go as deep as possible before backtracking, using recursion (or an explicit stack).

```
def dfs(graph, node, visited=None):
    if visited is None: visited = set()
    visited.add(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
    return visited
```

Time: $O(V + E)$ | **Space:** $O(V)$

9.3 Dijkstra's Algorithm (Shortest Path, non-negative weights)

```
import heapq

def dijkstra(graph, start):
    # graph: {node: [(neighbor, weight), ...]}
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    heap = [(0, start)]
    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]:
            continue
        for neighbor, weight in graph[node]:
            new_dist = d + weight
            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist
                heapq.heappush(heap, (new_dist, neighbor))
    return dist
```

Time: $O((V + E) \log V)$ with a heap | **Space:** $O(V)$

9.4 Bellman-Ford (handles negative weights, detects negative cycles)

```
def bellman_ford(edges, n, start):
    # edges: [(u, v, weight), ...]
    dist = [float('inf')] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    # Optional: one more pass to detect negative cycles
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            raise ValueError("Negative cycle detected")
    return dist
```

Time: $O(V \times E)$ | **Space:** $O(V)$

9.5 Floyd-Warshall (All-Pairs Shortest Path)

```
def floyd_warshall(matrix):
    # matrix[i][j] = weight, or inf if no edge
    n = len(matrix)
    dist = [row[:] for row in matrix]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

Time: $O(V^3)$ | **Space:** $O(V^2)$

9.6 Union-Find (Disjoint Set) — used for Kruskal's MST & cycle detection

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # path compression
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False  # already connected — would form a cycle
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        return True
```

9.7 Kruskal's Algorithm (Minimum Spanning Tree)

```
def kruskal(n, edges):  # edges: [(weight, u, v), ...]
    edges.sort()
    uf = UnionFind(n)
    mst_weight = 0
    for weight, u, v in edges:
        if uf.union(u, v):
            mst_weight += weight
    return mst_weight
```

Time: $O(E \log E)$ (dominated by sorting) | **Space:** $O(V)$

9.8 Prim's Algorithm (Minimum Spanning Tree)

```
import heapq

def prim(graph, start=0):  # graph: {node: [(neighbor, weight), ...]}
    visited = {start}
    heap = graph[start][:]
    heapq.heapify(heap)
    total_weight = 0
    while heap and len(visited) < len(graph):
        weight, node = heapq.heappop(heap)
        if node in visited:
            continue
        visited.add(node)
        total_weight += weight
        for neighbor, w in graph[node]:
            if neighbor not in visited:
                heapq.heappush(heap, (w, neighbor))
    return total_weight
```

Time: $O(E \log V)$ | **Space:** $O(V)$

9.9 Topological Sort (Kahn's Algorithm — BFS based)

```
from collections import deque

def topological_sort(graph, n):
    in_degree = [0] * n
    for node in graph:
        for neighbor in graph[node]:
            in_degree[neighbor] += 1

    queue = deque([i for i in range(n) if in_degree[i] == 0])
    order = []
    while queue:
```

```

node = queue.popleft()
order.append(node)
for neighbor in graph[node]:
    in_degree[neighbor] -= 1
    if in_degree[neighbor] == 0:
        queue.append(neighbor)

return order if len(order) == n else [] # empty = cycle detected

```

Time: $O(V + E)$ | **Space:** $O(V)$

🔖 Top 5 Graph Problems to Practice

1. Number of Islands
 2. Course Schedule
 3. Network Delay Time
 4. Clone Graph
 5. Word Ladder
-

PART 10: HASHING

Idea: A hash map/set gives you (on average) **$O(1)$** insert, delete, and lookup by converting a key into an array index using a **hash function**. It trades some memory for massive speed gains over linear search.

Collision handling (when two keys hash to the same index): -

Chaining: each bucket holds a linked list of entries. - **Open**

Addressing: on collision, probe for the next empty slot (linear/quadratic probing, or double hashing).

```

# Python's dict/set are hash maps/sets under the hood
seen = set()
freq = {}

for num in [1, 2, 2, 3]:
    freq[num] = freq.get(num, 0) + 1
    seen.add(num)

```

Classic use — Two Sum:

```

def two_sum(nums, target):
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

```

Time: $O(n)$ average (vs $O(n^2)$ brute force) | **Space:** $O(n)$

Time: Average $O(1)$ per operation | Worst $O(n)$ (all keys collide — rare with a good hash function)

🔖 Top 5 Hashing Problems to Practice

1. Two Sum
2. Group Anagrams
3. Longest Consecutive Sequence
4. Subarray Sum Equals K

PART 11: DYNAMIC PROGRAMMING (DP)

Idea: Break a problem into overlapping subproblems, solve each **once**, and reuse the answer (memoization) instead of recomputing. Applies when the problem has: - **Overlapping subproblems** — the same smaller problem shows up again and again. - **Optimal substructure** — the best overall answer is built from the best answers to subproblems.

How to approach any DP problem: 1. Define what `dp[i]` (or `dp[i][j]`) means in plain words. 2. Find the recurrence relating it to smaller states. 3. Identify base case(s). 4. Code it top-down (recursion + memo) or bottom-up (iterative table).

11.1 0/1 Knapsack

Idea: You have items with weight & value, and a bag of limited capacity. Pick items (each only once) to maximize value without exceeding capacity.

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            dp[i][w] = dp[i - 1][w] # don't take item i
            if weights[i - 1] <= w:
                dp[i][w] = max(dp[i][w], dp[i - 1][w - weights[i - 1]] + values[i - 1])
    return dp[n][capacity]
```

Time: $O(n \times \text{capacity})$ | **Space:** $O(n \times \text{capacity})$, reducible to $O(\text{capacity})$ with a rolling array

11.2 Longest Common Subsequence (LCS)

```
def lcs(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

Time: $O(m \times n)$ | **Space:** $O(m \times n)$

11.3 Longest Increasing Subsequence (LIS)

```
# O(n^2) DP version
def lis_dp(nums):
    n = len(nums)
```

```

dp = [1] * n
for i in range(n):
    for j in range(i):
        if nums[j] < nums[i]:
            dp[i] = max(dp[i], dp[j] + 1)
return max(dp) if dp else 0

# O(n log n) version using binary search (patience sorting)
import bisect
def lis_fast(nums):
    tails = []
    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)

```

Time: $O(n^2)$ basic | $O(n \log n)$ optimized | **Space:** $O(n)$

11.4 Coin Change (Unbounded Knapsack style)

```

def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for a in range(coin, amount + 1):
            dp[a] = min(dp[a], dp[a - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

```

Time: $O(\text{amount} \times \text{len}(\text{coins}))$ | **Space:** $O(\text{amount})$

DP on Trees (concept): Do a postorder traversal, returning results from children first, then combine at the parent (e.g., “House Robber III” — at each node decide rob-this-node vs skip-it, based on children’s answers).

Matrix Chain / Interval DP (concept): $dp[i][j]$ represents the best answer for the range $[i, j]$; iterate over a split point k between i and j and combine (e.g., “Burst Balloons”, “Matrix Chain Multiplication”).

🔖 Top 5 DP Problems to Practice

1. Longest Common Subsequence
2. 0/1 Knapsack
3. Coin Change
4. Longest Increasing Subsequence
5. Edit Distance

PART 12: GREEDY ALGORITHMS

Idea: Make the locally best choice at each step, hoping (and, for correct greedy problems, proving) it leads to the global optimum. Usually: sort first, then make one pass.

12.1 Activity Selection

```

def activity_selection(activities):
    # List of (start, end)
    activities.sort(key=lambda x: x[1])
    # sort by end time

```

```

count = 1
last_end = activities[0][1]
for start, end in activities[1:]:
    if start >= last_end:
        count += 1
        last_end = end
return count

```

Time: $O(n \log n)$ | **Space:** $O(1)$

12.2 Fractional Knapsack

```

def fractional_knapsack(items, capacity):    # items: [(value, weight), ...]
    items.sort(key=lambda x: x[0] / x[1], reverse=True) # by value/weight ratio
    total_value = 0
    for value, weight in items:
        if capacity >= weight:
            capacity -= weight
            total_value += value
        else:
            total_value += value * (capacity / weight)
            break
    return total_value

```

Time: $O(n \log n)$ | **Space:** $O(1)$

Huffman Coding (concept): Build an optimal prefix-free encoding by repeatedly merging the two lowest-frequency nodes into a new tree node (using a min-heap), until one tree remains. Used in compression (e.g., ZIP files).

🔖 Top 5 Greedy Problems to Practice

1. Jump Game
2. Gas Station
3. Activity Selection Problem
4. Huffman Encoding
5. Minimum Number of Platforms

PART 13: BIT MANIPULATION

Idea: Operate directly on the binary representation of numbers — extremely fast (hardware-level operations) and sometimes the only way to get $O(1)$ extra space.

Core operators to know:

```

a & b    # AND - 1 only if both bits are 1
a | b    # OR  - 1 if either bit is 1
a ^ b    # XOR - 1 if bits differ (great for toggling/finding unique elements)
~a       # NOT - flips all bits
a << 1   # left shift - multiply by 2
a >> 1   # right shift - divide by 2

```

Key tricks:

```

x & (x - 1) # clears the lowest set bit - used to count set bits
x & (-x)    # isolates the lowest set bit
x ^ x == 0  # a number XORed with itself is 0 - basis of "Single Number"

```

13.1 Count Set Bits

```
def count_bits(n):
    count = 0
    while n:
        n &= (n - 1)  # clears lowest set bit each time
        count += 1
    return count
```

Time: $O(\text{number of set bits})$ | **Space:** $O(1)$

13.2 Single Number (find the element that doesn't repeat)

```
def single_number(nums):
    result = 0
    for num in nums:
        result ^= num  # duplicates cancel out via XOR
    return result
```

Time: $O(n)$ | **Space:** $O(1)$

🕒 Top 5 Bit Manipulation Problems to Practice

1. Single Number
2. Counting Bits
3. Sum of Two Integers (without + or -)
4. Subsets (using bitmask)
5. Power of Two

PART 14: TRIE (PREFIX TREE)

Idea: A tree where each path from the root spells out a prefix of some word. Every node has children keyed by character. Lets you check “does any word start with X?” in $O(\text{length of X})$ — much faster than scanning every word.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end_of_word = True

    def search(self, word):
        node = self._find(word)
        return node is not None and node.is_end_of_word

    def starts_with(self, prefix):
        return self._find(prefix) is not None

    def _find(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
```

```
        return None
    node = node.children[ch]
    return node
```

Time: $O(L)$ per insert/search, where L = word length | **Space:** $O(\text{total characters across all inserted words})$

🔗 Top 5 Trie Problems to Practice

1. Implement Trie (Prefix Tree)
 2. Word Search II
 3. Design Add and Search Words Data Structure
 4. Replace Words
 5. Longest Word in Dictionary
-

Final Advice — How to Use This Handbook

1. **Don't read it front to back in one sitting.** Pick one topic, understand the idea, then type the code yourself (don't copy-paste) — that's what makes it stick.
2. **After each topic, solve at least 2 of the 5 listed problems** before moving to the next topic.
3. **When stuck on a new problem, come back and scan the "Idea" lines** — often the pattern-matching happens just from re-reading the plain-English explanation, not the code.
4. **Revisit topics after 1-2 weeks.** Spaced repetition is what turns "I understood this once" into "I recognize this instantly."

You've got the full map now — the rest is reps. Good luck!